Personal Statement

Yanghao Chen

My first research project at Tongji University involved fine-tuning a language model for mechanics of materials under the guidance of Professor Ying Zhao. I found datasets and Colab notebooks to learn from, but the mechanics content was limited and the data format did not match the training code. I studied the examples, prepared my own instruction and response pairs, and modified the notebook to read them. At first, I relied on examples to tell me what to do next. As I adapted the notebook, I worked out how the training code loaded and processed data, then adjusted it to work with my dataset. I learned to start with a specific problem, find the information I needed, and deepen my understanding by working on the code.

During the China International College Students' Innovation Competition, my teammates and I needed to deploy our fine-tuned model locally, but the available GPU memory was insufficient. After discussing possible solutions, we reduced the model's precision to lower its memory requirements and adjusted how it was saved and loaded, eventually deploying it locally. I learned to explain where I was stuck so that we could work through the problem together. Our team received a national-level award. Public datasets and notebooks had helped me get started, so I shared my own datasets and fine-tuned models on Hugging Face to provide similar resources for others.

Fine-tuning the language model left me wanting to understand more about neural networks and their use in mechanics. Continuing under Professor Ying Zhao's guidance, I studied conditional ConvLSTM models for data-driven prediction of lithium concentration and von Mises stress fields in battery materials. I used MATLAB scripts to run COMSOL simulations, arranged the concentration and stress images into training sequences in time order, and paired each sequence with its grain-orientation and C-rate maps. When examining successive predictions, I found that errors accumulated and the images gradually became distorted as predicted frames were fed back as inputs. I therefore introduced scheduled sampling to gradually include model-generated frames during training. I then compared multi-step autoregressive predictions with COMSOL reference sequences. Compared with training without scheduled sampling, image errors were lower over the same prediction horizon, and visible distortion appeared later. I became more careful about matching training conditions to actual use and checking predictions over multiple steps. Training these models required labeled data generated by simulations. I began to ask whether physical laws could be used directly to train a neural network without precomputed reference solutions.

I explored this question under Professor Xianyang (Tom) Chen's guidance, using physics-informed neural networks (PINNs) for the one-dimensional inviscid Burgers equation. I used automatic differentiation in PyTorch to compute the derivatives needed for the partial differential equation (PDE) residual, then trained the network with residual, initial-condition, and boundary-condition losses rather than precomputed solution labels. Capturing the sharp shock transition was difficult, so I tested an artificial-viscosity term with a fixed coefficient in the residual. Against the analytical inviscid solution, this model had a lower relative L^2

error but a higher maximum pointwise error than the standard PINN. I examined the shock location and transition width to understand the difference. Adding artificial viscosity also changed the equation being solved, so a lower error alone was not enough to conclude that the method was better. My PINN was trained for a fixed set of initial and boundary conditions; changing them would require retraining. I wanted to know whether a network could learn the relationship between these conditions and the solution, so that the same trained model could handle different inputs.

This question led me to neural operators, which learn function-to-function mappings. A trained operator can be reused for different inputs within a class of problems, without retraining for each input. I wanted to explore how this reuse could support repeated computations in solid mechanics. At Johns Hopkins University, I pursued this direction under the supervision of Professor Somdatta Goswami through a hybrid finite element–neural operator solver. Using data generated with GPU-accelerated JAX-FEM, I trained Transolver in JAX to map interface displacement fields to subdomain stress fields. Within a non-overlapping domain decomposition framework, I coupled the neural operator with FEM and reused the trained operator as interface displacements changed. However, stress prediction errors were small on held-out Gaussian random field (GRF) samples but much larger for displacements produced during coupling. I compared the GRF-sampled displacements with the actual coupling inputs and found a mismatch between their distributions. To address this distribution shift, I adjusted the GRF length scale and amplitude and generated new training data. After retraining, prediction errors decreased in coupling tests on cylinder and bracket problems. The predicted stresses are used to compute interface reaction forces passed to FEM, which influence the next displacement update, so stress prediction errors need to be considered within the full iterative process. Alongside stress accuracy, I checked interface displacement compatibility, force balance, and convergence. I am now extending this work to three-dimensional fiber-reinforced composites, using FEM for the matrix and sharing neural-operator weights across fiber subdomains.

During my PhD, I want to develop methods that combine neural operator learning with numerical simulation. My experience with JAX and JAX-FEM has drawn me toward GPU-accelerated scientific computing and differentiable programming for solid mechanics. I would like to explore how neural operators and numerical solvers can be integrated into differentiable computational graphs to support end-to-end gradient computation. To pursue these questions, I hope to deepen my understanding of numerical methods and machine learning and gain more experience developing and testing computational methods. In a doctoral group, I hope to discuss problems openly, share code and data, and help others when I can, just as my teammates and mentors have helped me.